

2. Node and Graph Features

2026-03-11, 21:18

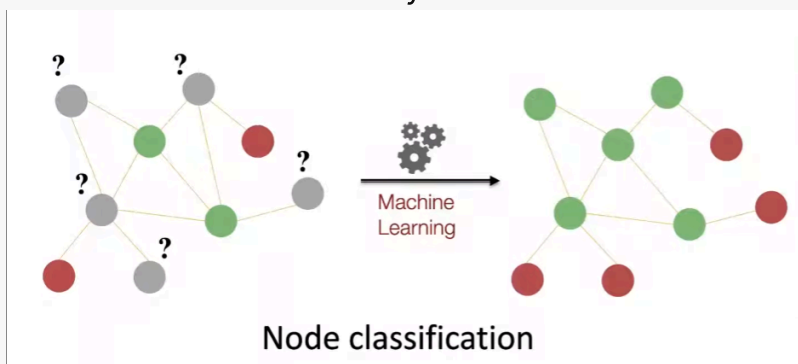
Tags: [GNN Deep Learning](#)

Node-level Tasks and Features

Given : $G = (V, E)$

Find function : $f : v \rightarrow R$

In this case, we know the few nodes that are labeled, and we are finding the nodes that are not labeled yet.



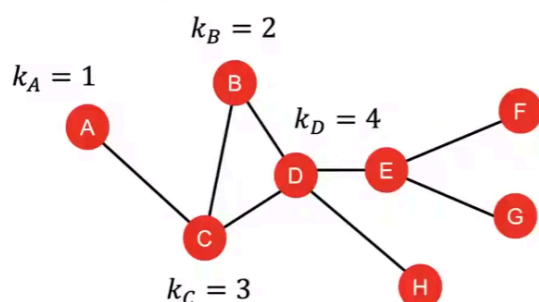
Correctly color the node here.

Therefore, we need *Node-Level features*:

- Node degree
- Node centrality
- Clustering coefficient
- Graphlets

Node Degree

- The degree k_v of node v is the number of edges (neighboring nodes) the node has.
- Treats all neighboring nodes equally.



****Key Point:** it treat all neighboring nodes equally

Node Centrality

Node centrality c_v takes the node importance in a graph into account:

- **Different ways to model importance:**

- Eigenvector centrality
- Betweenness centrality
- Closeness centrality
- and many others...

Eigenvector centrality

- A node v is important if surrounded by important neighboring nodes $u \in N(v)$

- **Eigenvector centrality:**

- A node v is important if surrounded by important neighboring nodes $u \in N(v)$.

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u$$

λ is some positive constant

$$\lambda \mathbf{c} = \mathbf{A} \mathbf{c}$$

- $\mathbf{A}$: Adjacency matrix
 $A_{uv} = 1$ if $u \in N(v)$
- $\mathbf{c}$: Centrality vector

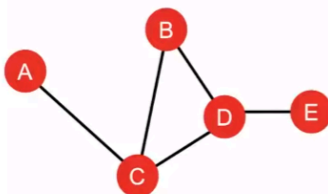
- The largest eigenvalue λ_{max} is always positive and unique (by Perron-Frobenius Theorem).
- The leading eigenvector $\mathbf{c}_{max}$ is used for centrality.

Betweenness Centrality

- A node is important if it lies on many shortest paths between other nodes

$$c_v = \sum_{s \neq v \neq t} \frac{\#(\text{shortest paths between } s \text{ and } t \text{ that contain } v)}{\#(\text{shortest paths between } s \text{ and } t)}$$

- **Example:**



$$\begin{aligned} c_A &= c_B = c_E = 0 \\ c_C &= 3 \\ &(\text{A-C-B, A-C-D, A-C-D-E}) \\ c_D &= 3 \\ &(\text{A-C-D-E, B-D-E, C-D-E}) \end{aligned}$$

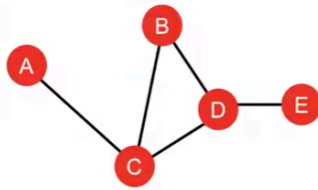
Evaluating the importance of the transit hub of the node

Closeness Centrality

- A node is important if it has small shortest path lengths to all other nodes

$$c_v = \frac{1}{\sum_{u \neq v} \text{shortest path length between } u \text{ and } v}$$

■ **Example:**



$$c_A = 1/(2 + 1 + 2 + 3) = 1/8$$

(A-C-B, A-C, A-C-D, A-C-D-E)

$$c_D = 1/(2 + 1 + 1 + 1) = 1/5$$

(D-C-A, D-B, D-C, D-E)

Clustering Coefficient

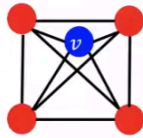
- It measures how connected v 's neighboring nodes are:

- Measures how connected v 's neighboring nodes are:

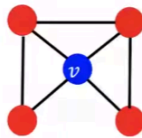
$$e_v = \frac{\#(\text{edges among neighboring nodes})}{\binom{k_v}{2}} \in [0,1]$$

#(node pairs among k_v neighboring nodes)

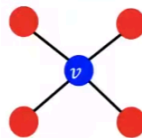
- Examples:



$$e_v = 1$$



$$e_v = 0.5$$



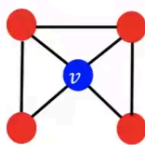
$$e_v = 0$$

- Since k_v is the degree, which represents the number of neighbors node v has, k_v choose 2 represents the maximum possible number of edges that could exist between v 's neighbors. (represents how many distinct pairs you can form from n items)
- The ratio is between 0 and 1.

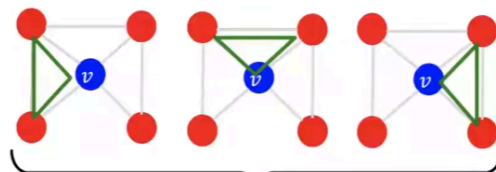
eg. the $e_v = 0.5$ cases. the edges around the nodes = 3; $k_v = 4$. Thus $4 \text{ choose } 2 = 6$. $3/6 = 0.5$

Graphlets

- Observation : Clustering coefficient counts the #(triangles) in the ego-network



$$e_v = 0.5$$

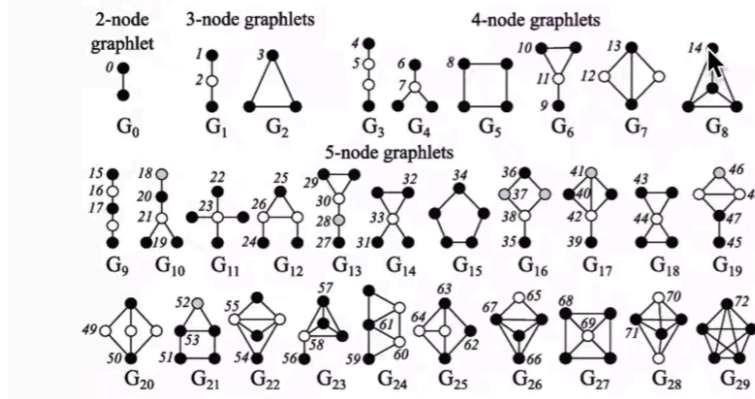


3 triangles (out of 6 node triplets)

Definition: Graphlets rooted connected non-isomorphic subgraphs

- However, there are different graphs, and also, the position in the graph. (Different positions are not equal to each other, thus creating new graphlets) → 72 in total

Graphlets: Rooted connected non-isomorphic subgraphs:



*Orbit: An orbit is defined using a **Graph Automorphism**. An automorphism is a permutation (reordering) of the nodes that preserves the adjacency of the graph.*

- Graphlet Degree Vectors

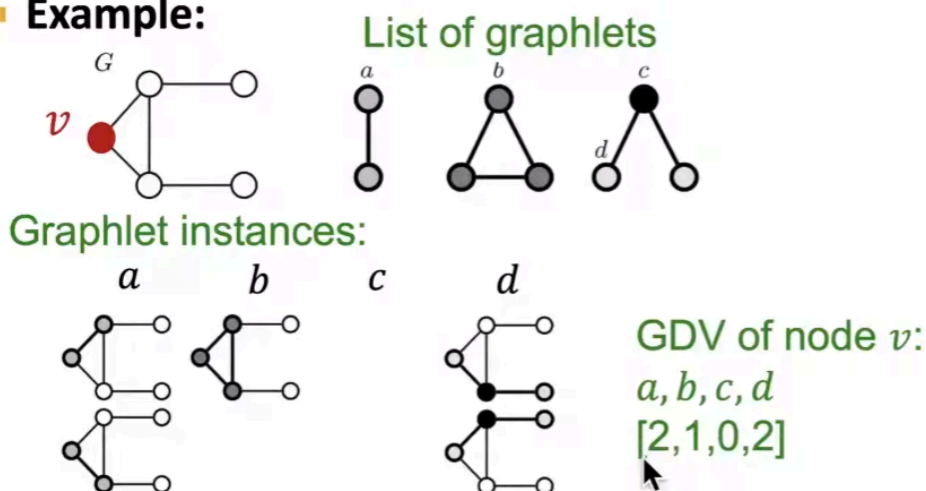
Graphlet Degree Vector: Graphlet-base features for nodes

Counts the number of graphlet that are touching.

eg.

- Graphlet Degree Vector (GDV):** A count vector of graphlets rooted at a given node.

- Example:**



In the Graphlet Degree Vector (GDV) calculation, we don't just ask "Is this shape present?";

we ask "**Does node v play the role of the black dot in this shape?**"

- Graphlet c has the root (black dot) on the middle node. (because the neighboring point must be connected)
- Graphlet d has the root (black dot) on an end node.

- Considering graphlets on 2 to 5 nodes we get:
 - **Vector of 73 coordinates** is a signature of a node that describes the topology of node's neighborhood
 - Captures its interconnectivities out to a **distance of 4 hops**
- Graphlet degree vector provides a measure of a **node's local network topology**:
 - Comparing vectors of two nodes provides a more detailed measure of local topological similarity than node degrees or clustering coefficient.

Link Prediction Task and Features

The task is: link prediction, predict new links based on existing link in the network

The key is to design features for a pair of nodes.

Two formulations of the link prediction task

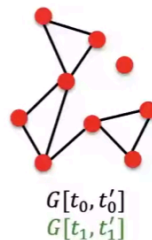
Two formulations of the link prediction task:

■ 1) Links missing at random:

- Remove a random set of links and then aim to predict them

■ 2) Links over time:

- Given $G[t_0, t'_0]$ a graph on edges up to time t'_0 , **output a ranked list L** of links (not in $G[t_0, t'_0]$) that are predicted to appear in $G[t_1, t'_1]$



■ Evaluation:

- $n = |E_{new}|$: # new edges that appear during the test period $[t_1, t'_1]$
- Take top n elements of L and count correct edges

• Methodology:

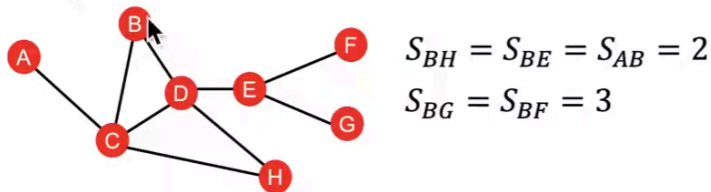
1. For each pair of nodes (x, y) compute score $c(x, y)$
2. Sort pairs (x, y) by the decreasing score $c(x, y)$
3. Predict top n pairs as new links

Link-level features

1. Distance Based features

This is finding the shortest path distance between two nodes

- Example:



- However, this does not capture the degree of neighborhood overlap:
 - Node pair (B, H) has 2 shared neighboring nodes, while pairs (B, E) and (A, B) only have 1 such node.

2. Local Neighborhood Overlap

Captures # neighboring nodes shared between two nodes v_1 and v_2 :

- **Common neighbors:** $|N(v_1) \cap N(v_2)|$

- Example: $|N(A) \cap N(B)| = |\{C\}| = 1$

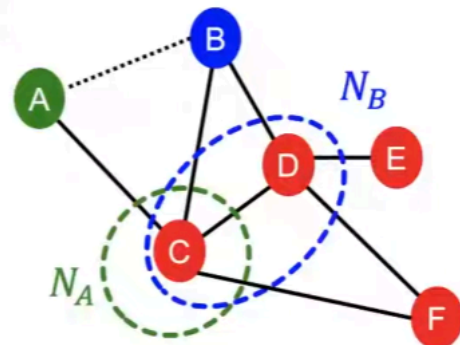
- **Jaccard's coefficient:** $\frac{|N(v_1) \cap N(v_2)|}{|N(v_1) \cup N(v_2)|}$

- Example: $\frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|} = \frac{|\{C\}|}{|\{C, D\}|} = \frac{1}{2}$

- **Adamic-Adar index:**

$$\sum_{u \in N(v_1) \cap N(v_2)} \frac{1}{\log(k_u)}$$

- Example: $\frac{1}{\log(k_C)} = \frac{1}{\log 4}$



You can find either:

- Common neighbors (the union of the neighboring nodes)
- Jaccard's coefficient (common neighbors over the total degree)
- Adamic-Adar index (and log operations)

But there is one limitation: two nodes may still potentially be connected in the future even without local overlaps

3. Global Neighborhood overlap

Katz Index: count the number of paths of all lengths between a given pair of nodes

The representation: $P_{ab}^{(x)}$, which represents *the number of path of length x that starts at node a and ends at node b .*

How to compute $P_{uv}^{(2)}$?

- Step 1: Compute **#paths** of length 1 **between each of u 's neighbor and v**

- Step 2: **Sum up** these #paths across u 's neighbors

$$P_{uv}^{(2)} = \sum_i A_{ui} * P_{iv}^{(1)} = \sum_i A_{ui} * A_{iv} = A_{uv}^2$$

Node 1's neighbors #paths of length 1 between Node 1's neighbors and Node 2 $P_{12}^{(2)} = A_{12}^2$

$$A^2 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \times \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}$$

Power of adjacency

Since it is $u \rightarrow v$; thus you need to first get to u , then get to v . So it is the multiplication of itself to get the entry multiple times with ones.

Therefore, the Katz Index will do the job:

- Katz index:** count the number of paths of all lengths between a pair of nodes.
- How to compute #paths between two nodes?
 - Use **adjacency matrix powers!**
 - A_{uv} specifies #paths of length 1 (direct neighborhood) between u and v .
 - A_{uv}^2 specifies #paths of **length 2** (neighbor of neighbor) between u and v .
 - And, A_{uv}^l specifies #paths of **length l** .

- Length 1 (A): Direct friends.
- Length 2 (A^2): Friends-of-friends.
- Length l (A^l): l -degree of separation.

Then, we find the sum across all the path lengths, where Katz index matrix is computed in closed-form:

- **Katz index** between v_1 and v_2 is calculated as

Sum over all path lengths

$$S_{v_1 v_2} = \sum_{l=1}^{\infty} \beta^l A_{v_1 v_2}^l$$

#paths of length l
between v_1 and v_2

$0 < \beta < 1$: discount factor

- Katz index matrix is computed in closed-form:

$$S = \sum_{i=1}^{\infty} \beta^i A^i = (I - \beta A)^{-1} - I,$$

$= \sum_{i=0}^{\infty} \beta^i A^i$
by geometric series of matrices

Graph-Level Features and Graph Kernels

Design kernels instead of feature vectors

- A quick intro to kernels:
 1. Kernel $K(G, G') \in \mathbb{R}$ measures similarity between data sets
 2. Kernel matrix $K = (K(G, G'))_{G, G'}$ must always be positive semidefinite (i.e., has positive eigenvals)
 3. There exists a feature representation such that $K(G, G') = (G)^T(G')$
 4. Once the kernel is defined, off-the-shelf ML model, such as **kernel SVM**, can be used to make predictions.

Goal: Design graph feature vector (G)

Key idea: **Bag-of-Words (BoW) for a graph**

- **Recall:** BoW simply uses the word counts as features for documents (no ordering considered).
- Naïve extension to a graph: **Regard nodes as words.**
- Since both graphs have **4 red nodes**, we get the same feature vector for two different graphs...

$$\phi(\text{Graph 1}) = \phi(\text{Graph 2})$$

(But the bag of words cannot effectively represent the node information)

Thus, we use **Bag of node degrees**

What if we use Bag of node degrees?

Deg1: ● Deg2: ● Deg3: ●

$$\phi(\text{graph}) = \text{count}(\text{graph}) = [1, 3, 0]$$

Obtains different features for different graphs!

$$\phi(\text{graph}) = \text{count}(\text{graph}) = [0, 2, 2]$$

- Both Graphlet Kernel and Weisfeiler-Lehman (WL) Kernel use **Bag-of-*** representation of graph, where * is more sophisticated than node degrees!

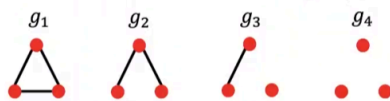
Graphlet Kernels

Count the number of different graphlets in a graph

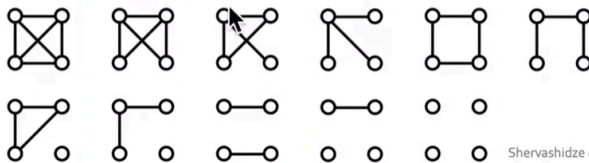
- The graphlet in the graphlet kernels are different from the kernel in the node level:

Let $\mathcal{G}_k = (g_1, g_2, \dots, g_{n_k})$ be a list of graphlets of size k .

- For $k = 3$, there are 4 graphlets.



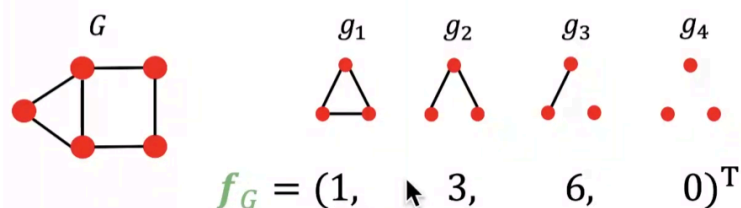
- For $k = 4$, there are 11 graphlets.



Shervashidze et al., AISTATS 2011

- The Mathematical Notation used:
 - G : The large graph you are analyzing.
 - k : A pre-defined list of all possible graphlets (shapes) with k nodes.
 - $f_G \in \mathbb{R}^{n_k}$: This means the result is a vector (a list of numbers) where each number corresponds to a count of one specific shape.
 - $(f_G)_i = (g_i \text{ in } G)$: This simply means the i -th number in your vector is the number of times graphlet g_i appears as a subgraph in G .

- Given graph G , and a graphlet list $\mathcal{G}_k = (g_1, g_2, \dots, g_{n_k})$, define the graphlet count vector $f_G \in \mathbb{R}^{n_k}$ as $(f_G)_i = \#(g_i \subseteq G)$ for $i = 1, 2, \dots, n_k$.
- Example for $k = 3$



Therefore, for any given two graphs, G and G' , graphlet kernel is computed as

$$K(G, G') = f_G^T \times f_{G'}$$

But for this, G and G' might have different sizes, that will greatly skew the value.

Solution: : we normalize each feature vector

$$h_G = \frac{f_G}{\text{sum}(f_G)}$$

$$K(G, G') = h_G^T \times h_{G'}$$

- Computational Limit

Limitations: Counting graphlets is **expensive!**

- Counting size- k graphlets for a graph with size n by enumeration takes n^k .
- This is unavoidable in the worst-case since **subgraph isomorphism test** (judging whether a graph is a subgraph of another graph) is **NP-hard**.
- If a graph's node degree is bounded by d , an $O(nd^{k-1})$ algorithm exists to count all the graphlets of size k .

Can we design a more efficient graph kernel?

Weisfeiler-Lehman Kernel

The idea here is to use neighborhood structure to iteratively enrich node vocabulary

Color Refinement

- **Given:** A graph G with a set of nodes V .
 - Assign an initial color $c^{(0)}(v)$ to each node v .
 - Iteratively refine node colors by

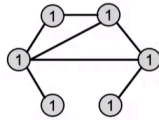
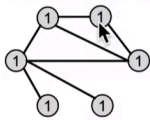
$$c^{(k+1)}(v) = \text{HASH} \left(\left\{ c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)} \right\} \right),$$
 where **HASH** maps different inputs to different colors.
 - After K steps of color refinement, $c^{(K)}(v)$ summarizes the structure of K -hop neighborhood

In steps:

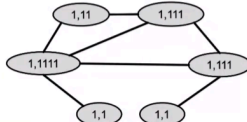
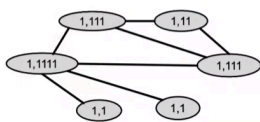
1. Color refinement (Assign the initial colors and aggregate neighboring colors)

Example of color refinement given two graphs

- Assign initial colors



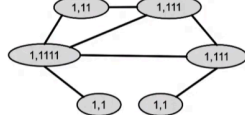
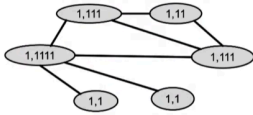
- Aggregate neighboring colors



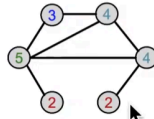
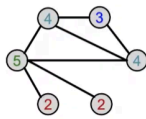
2. Apply hash function to produce the new color

Example of color refinement given two graphs

- Aggregated colors



- Hash aggregated colors



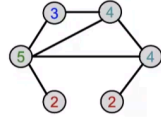
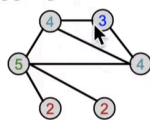
Hash table

1,1	→	2
1,11	→	3
1,111	→	4
1,1111	→	5

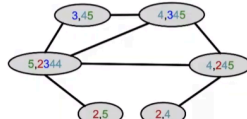
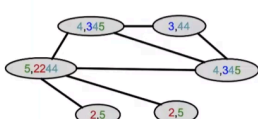
3. Hash the aggregated Color again

Example of color refinement given two graphs

- Aggregated colors



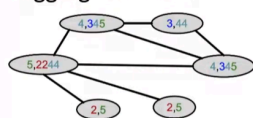
- Hash aggregated colors



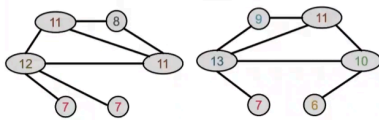
4. Run it till you want to stop

Example of color refinement given two graphs

Aggregated colors



Hash aggregated colors



Hash table

2,4	→	6
2,5	→	7
3,44	→	8
3,45	→	9
4,245	→	10
4,345	→	11
5,2244	→	12
5,2344	→	13

After color refinement, WL kernel counts number of nodes with a given color

After color refinement, WL kernel counts number of nodes with a given color.

$$\phi(\text{Graph 1}) = \begin{matrix} \text{Colors} \\ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 \\ \text{Counts} \\ [6, 2, 1, 2, 1, 0, 2, 1, 0, 0, 0, 2, 1] \end{matrix}$$

$$\phi(\text{Graph 2}) = \begin{matrix} \text{Colors} \\ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 \\ \text{Counts} \\ [6, 2, 1, 2, 1, 1, 1, 0, 1, 1, 1, 0, 1] \end{matrix}$$

Then do the dot product with the labeled colors.

The WL kernel value is computed by the inner product of the color count vectors:

$$\begin{aligned} K(\text{Graph 1}, \text{Graph 2}) &= \phi(\text{Graph 1})^T \phi(\text{Graph 2}) \\ &= 49 \end{aligned}$$

It is computationally efficient, because the computational process is linear

Short Summary

Graphlet Kernel

- Graph is represented as **Bag-of-graphlets**

- Computationally expensive**

Weisfeiler-Lehman Kernel

- Apply K -step color refinement algorithm to enrich node colors
 - Different colors capture different K -hop neighborhood structures
- Graph is represented as **Bag-of-colors**
- Computationally efficient**
- Closely related to Graph Neural Networks (as we will see!)

Reference

<https://www.youtube.com/watch?v=buzsHTa4Hgs&list=PL-myakI4DsIVPlos0HopgT8SjOn7HLJw8&index=27>